ITSAR2 313 – SYSTEM INTEGRATION AND ARCHITECTURE 2

LABORATORY REPORT: EXERCISE 2 — MICROSERVICES EDGE CASE TESTING

Gayuma, Jeremiah; Mamar, Kharl Benedict; Alacapa, Jesryl John; Barbas, Christian; Orbigoso, Mikhoel Runo
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory exercise focused on designing and implementing a Product Ordering API using Node.js and Express that enforces clear business rules over product inventory and shopping cart operations. By applying a three-layer architecture (presentation layer for HTTP requests and views, business logic layer for validation and rules, and data layer for product persistence), the system allows users to browse products, manage a session-based shopping cart, and place orders either directly or via checkout. Throughout the exercise, attention was given to validating inputs such as product existence, quantity constraints, and stock availability before any update is committed. Systematic testing using browser-based pages and curl commands in the terminal confirmed that valid requests are processed correctly, error conditions are handled gracefully with descriptive messages, and product stock and cart contents remain consistent. The overall outcome is a working order-processing system that demonstrates how properly structured business logic and well-defined endpoints can support reliable, rule-driven transactions.

The exercise provided a hands-on application of theoretical concepts related to web systems, particularly the separation of concerns and the role of business logic in transactional applications. Implementing the Product Ordering API required understanding how data flows from client requests through Express routes, into validation functions, and ultimately to the data store that reflects stock changes. Working with a session-based cart highlighted the importance of maintaining user-specific state across multiple requests, such as browsing, adding items, updating quantities, and checking out. Using curl to test each endpoint emphasizes how APIs are consumed not only by browsers but also by scripts and other services, reinforcing the need for consistent JSON responses and meaningful HTTP status codes. Overall, the exercise deepened my understanding of how backend components collaborate to enforce real-world rules, such as preventing over-ordering or selling out-of-stock items, which are critical for dependable e-commerce systems.

I. INTRODUCTION

This laboratory activity is anchored on several key concepts: system architecture, business logic enforcement, and RESTful API design. A central theory applied is the separation of concerns, where responsibilities are divided among distinct layers. The presentation layer is responsible for handling HTTP requests and responses, including rendering HTML pages for the shop, cart, checkout, and order confirmation views. The business logic layer contains the rules that govern ordering behavior, such as validating that products exist, requiring positive integer quantities, preventing orders for zero-stock items, and ensuring that requested quantities do not exceed available stock. The data layer maintains the list of products and their current stock levels, reflecting updates after successful orders. Additionally, RESTful

principles are observed through clear resource-oriented endpoints (for example, product listing, cart operations, and order placement) and the use of appropriate HTTP methods (GET for retrieval and POST for state changes). Session management is another important concept in this lab, as it is used to maintain each user's cart across multiple requests. Together, these theories and requisites provide the foundation for achieving the main objective of the exercise, which is to build a robust product ordering system that behaves predictably and safely under various usage scenarios.

II. METHODOLOGY / IMPLEMENTATION

The implementation began by initializing a Node.js project and configuring an Express server as the main entry point. Middleware was set up to parse JSON and URL-encoded request bodies, enabling the server to accept data from both browser forms and programmatic clients. An express-session middleware was then added to maintain a per-user shopping cart stored on the server, ensuring that cart operations (add, update, remove, clear) are tied to an individual session. The data layer was implemented using a products.json file containing product IDs, names, prices, stock levels, and categories. Helper functions were written to load this product list from disk and to save updated stock values back to the file whenever a successful order changes inventory.

On top of this, the business logic layer was implemented through functions that centralize validation rules. For direct orders, a validator checks that the product exists, that the quantity is a positive integer, that the product is not out of stock, and that the requested quantity does not exceed available stock. For cart-based checkouts, a separate function traverses all items in the cart and confirms that each product still exists and has enough remaining stock to fulfill the requested quantities. Controllers were then organized by responsibility: product controllers handle listing all products, fetching single products, and creating direct orders; cart controllers manage adding items, updating quantities, removing items, clearing the cart, and placing orders from the cart; view controllers render HTML pages (shop, cart, checkout, and confirmation) using a shared layout.

1. Get all products (Figure: GET /products)

   curl.exe -i "http://localhost:3000/products"

```
PS C:\laragon\www\ITPFL> curl.exe -i "http://localhost:3000/products"
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 819
ETag: W/"333-4Mu9prbntXZZB2d500/Na6nQI3Q"
Set-Cookie: connect.sid=s%3A-5q0SbgVl2bHRS96QVBQ8zrwA3SYGesj.NrKFDmCNF2eManzBjeqk8T3Zih1O3J7h2qjg7McQl5g; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 16:22:15 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"data":[{"id":1,"name":"Laptop","price":999.99,"stock":8,"category":"Prebuilt PC","image":"https://images.unsplash.com/phot
o-1496181133206-80ce9b88a853?w=600"},{"id":2,"name":"Wireless Mouse","price":29.99,"stock":49,"category":"Peripheral","image":"https://imag
es.unsplash.com/photo-1527864550417-7fd91fc51a46?w=600"},{"id":3,"name":"Mechanical Keyboard","price":59.99,"stock":0,"category":"Periphera
l","image":"https://images.unsplash.com/photo-1618384887929-16ec33fab9ef?w=600"},{"id":4,"name":"27\" Gaming Monitor","price":399.99,"stock
":0,"category":"Monitor","image":"https://images.unsplash.com/photo-1527443224154-c4a3942d3acf?w=600"},{"id":5,"name":"Gaming Headset","pri
ce":79.99,"stock":20,"category":"Peripheral","image":"https://images.unsplash.com/photo-1599669454699-248893623440?w=600"}]}
```

2. Get single product (Figure: GET /products/1)

curl.exe -i "http://localhost:3000/products/1"

```
PS C:\laragon\www\ITPFL> curl.exe -i "http://localhost:3000/products/1"
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 175
ETag: W/"af-wwVZIjBsvbAVHbbWyjQxJvPfS1Y"
Set-Cookie: connect.sid=s%3AwEI3aHE9y_euOduFicPkomf8E54qrJFN.D%2BdS7g3xewbGklM4WQ%2BcKWRbFhnD8gAvua9oXc6vSLw; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 16:00:50 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"data":{"id":1,"name":"Laptop","price":999.99,"stock":8,"category":"Prebuilt PC","image":"https://images.unsplash.com/photo-1496181133206-80ce9b8
8a853?w=600"}}
```

3. Add first item to cart + save session cookie (Figure: POST /cart/add #1)

curl.exe -i -X POST "http://localhost:3000/cart/add" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept: application/json" -d "productId=1&quantity=1" -c cookies.txt

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/cart/add" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept:
 application/json" -d "productId=1&quantity=1" -c cookies.txt
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 88
ETag: W/"58-b3FcZnfDs5SccwRnpej5+VfnQUE"
Set-Cookie: connect.sid=s%3AZaVB8q1G7gSJ1xgsOlBQ4DkCfjbiekPU.NMNQbPR52Us2CcpQi%2FDmJF46q8RoWhKxo%2BGe%2BPzGiQQ; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 17:08:56 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"message":"Laptop added to cart!","cart":[{"productId":1,"quantity":1}]}
```

4. Add second item to cart (Figure: POST /cart/add #2)

curl.exe -i -X POST "http://localhost:3000/cart/add" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept: application/json" -d "productId=2&quantity=3" -b cookies.txt -c cookies.txt

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/cart/add" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept:
 application/json" -d "productId=2&quantity=3" -b cookies.txt -c cookies.txt
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 125
ETag: W/"7d-5qf2y+lKb7DVfnLCiwj9IssaoLk"
Date: Tue, 24 Mar 2026 17:09:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"message":"Wireless Mouse added to cart!","cart":[{"productId":1,"quantity":1},{"productId":2,"quantity":3}]}
```

5. View cart contents (Figure: GET /cart/api)

curl.exe -i "http://localhost:3000/cart/api" -H "Accept: application/json" -b cookies.txt

```
PS C:\laragon\www\ITPFL> curl.exe -i "http://localhost:3000/cart/api" -H "Accept: application/json" -b cookies.txt
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 358
ETag: W/"166-BgS3zb6TG1RdYLsat2Z7dkTyq3w"
Date: Tue, 24 Mar 2026 17:10:24 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"items":[{"productId":1,"quantity":1,"name":"Laptop","price":999.99,"image":"https://images.unsplash.com/photo-1496181133206-80ce9b88a853?w=600","su
btotal":999.99},{"productId":2,"quantity":3,"name":"Wireless Mouse","price":29.99,"image":"https://images.unsplash.com/photo-1527864550417-7fd91fc51a46?w=600","subt
otal":89.97}],"total":1089.96}
```

6. Update quantity in cart (Figure: POST /cart/update)

   curl.exe -i -X POST "http://localhost:3000/cart/update" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept: application/json" -d "productId=1&quantity=2" -b cookies.txt -c cookies.txt

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/cart/update" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept: application/json"
 -d "productId=1&quantity=2" -b cookies.txt -c cookies.txt
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 83
ETag: W/"53-AQhWBDZsCXHMC3EfyRaFm5OzIFs"
Date: Tue, 24 Mar 2026 17:12:37 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"cart":[{"productId":1,"quantity":2},{"productId":2,"quantity":3}]}
```

7. Remove an item from cart (Figure: POST /cart/remove)

   curl.exe -i -X POST "http://localhost:3000/cart/remove" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept: application/json" -d "productId=2" -b cookies.txt -c cookies.txt

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/cart/remove" -H "Content-Type: application/x-www-form-urlencoded" -H "Accept: application/jso
 -d "productId=2" -b cookies.txt -c cookies.txt
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 80
ETag: W/"50-BlHckLOdLrmUIk8Lra2/v2SckEY"
Date: Tue, 24 Mar 2026 17:13:45 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"message":"Item removed.","cart":[{"productId":1,"quantity":2}]}
```

8. Place order from cart (Figure: POST /checkout/place-order)

   curl.exe -i -X POST "http://localhost:3000/checkout/place-order" -H "Accept: application/json" -b cookies.txt

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/checkout/place-order" -H "Accept: application/json" -b cookies.txt
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 214
ETag: W/"d6-EbTB2dk6YEqKhzoebd8PoroYruw"
Date: Tue, 24 Mar 2026 17:14:26 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"message":"Order placed successfully!","order":{"items":[{"name":"Laptop","price":999.99,"quantity":2,"image":"https://images.unsplash.com/photo-149
6181133206-80ce9b88a853?w=600"}],"total":1999.98}}
```

9. Direct order (bypasses cart) (Figure: POST /orders)

   curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=1&quantity=2"
   Expected: HTTP/1.1 200 OK with JSON

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=1&quantity=2"
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 137
ETag: W/"89-ZqitO5Fhuhf8X4xq3EYlknYJaDM"
Set-Cookie: connect.sid=s%3AmAMj2NKPXQ3VUQHdw3FA3XAk2kpkH7tc.9a8xzyP3uQLQ85s35%2Fh1p28f5P0t9OmI1h9jkEwqsLc; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 17:15:02 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":true,"message":"Order placed successfully!","order":{"product":"Laptop","quantity":2,"totalPrice":1999.98,"remainingStock":4}}
```

In addition, a set of error-focused curl tests was performed and documented with screenshots to support the experimental and discussion sections:

10 Product not found

curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=9999&quantity=1"

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=9999&quantity=1"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 48
ETag: W/"30-sUunr4C7wbybN8l0x/eny8w07NE"
Set-Cookie: connect.sid=s%3AT1jocmmHJDDLg-5uav5tYYpzeWk32gmP.8p4rMbYQ8RpdqBxEKchvHdksEsoMpq6wSoz9VcNjtco; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 17:16:18 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":false,"message":"Product not found."}
```

11 Out of stock

curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=3&quantity=1"

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=3&quantity=1"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 66
ETag: W/"42-KAv3U1WQ5Jl2fz9qDW5Sc5lXn9c"
Set-Cookie: connect.sid=s%3AtAqtvYpF4hcPPIJU89gEv_wtaCpg6YiL.MzZhtwNaetn5bpU7%2BAk2JewRdb7qodpRPScdMHEhqxE; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 17:17:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":false,"message":"Mechanical Keyboard is out of stock."}
```

12 Invalid quantity

curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=1&quantity=0"

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=1&quantity=0"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 66
ETag: W/"42-6z8nIT3CfzyQDFz//WeOJ/ddqGc"
Set-Cookie: connect.sid=s%3AFO5mZFqHmD6Lf30Od0fJvuasLkhTd-nl.VSdgXmlux4GBy3Y%2BXTuNpaOVGCry9pdFSrLnbImhMxw; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 17:17:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":false,"message":"Quantity must be a positive integer."}
```

13 Missing fields

curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=1"

```
PS C:\laragon\www\ITPFL> curl.exe -i -X POST "http://localhost:3000/orders" -H "Content-Type: application/x-www-form-urlencoded" -d "productId=1"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 66
ETag: W/"42-23KQYK50TPH7VgQf3rtzsTUaujI"
Set-Cookie: connect.sid=s%3ANT6j3u2FOgzLFyKVJd35Bvdqk9syNZHj.JkdZmv4A1H7%2F%2BFrkvdQPwBISQnjcRLF4iDgNhb5fSs4; Path=/; HttpOnly
Date: Tue, 24 Mar 2026 17:18:21 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"success":false,"message":"productId and quantity are required."}
```

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

The test scenarios for the TechVault product-ordering system produced the expected HTTP status codes and JSON responses. The normal-flow tests showed that the API can reliably list all products, retrieve a specific product by ID, accept a valid order, and immediately reflect stock reductions in subsequent GET requests. Cart-based operations also behaved correctly: items could be added with different quantities, viewed through the cart API, updated, removed, and finally checked out to create an order while clearing the cart and updating inventory.

The remaining scenarios focused on invalid input and error handling. Requests using non-existent product IDs returned “Product not found” with the appropriate 4xx status. Zero, negative, and overly large quantities were rejected as invalid or exceeding stock, and missing required fields such as productId or quantity produced validation errors rather than silent failures. An intentionally incorrect route returned a JSON 404 response instead of a generic HTML error page. Together, these results indicate that the application enforces its business rules consistently and fails safely when clients send incorrect or incomplete data.

IV. LAB DISCUSSIONS / COMPUTATIONS

The lab showed that the TechVault ordering system behaves correctly for both normal use and error conditions. When valid requests were sent, the API listed products, retrieved items by ID, accepted orders, and updated the remaining stock in a way that matched what we expected based on the quantities ordered. Cart operations also worked as intended: items could be added, viewed, updated, removed, and checked out, and the responses clearly reflected the current contents of the cart and the final order details.

For invalid requests, the system responded with clear error messages instead of silently failing. Incorrect product IDs, bad or missing quantities, and attempts to order more items than are in stock all produced appropriate 4xx status codes and descriptive JSON responses. An intentionally wrong URL also returned a proper “not found” response. Overall, the behavior observed in testing confirms that the core logic for pricing, inventory updates, and validation is consistent and reliable.

V. CONCLUSIONS

The lab successfully achieved its objective of implementing and validating a Product Ordering API that enforces realistic business rules over product inventory and customer orders. By structuring the system into presentation, business logic, and data layers, the implementation maintained a clear separation of concerns, making it easier to reason about

how requests are processed and how rules are enforced. Centralized validation logic ensured that constraints on product existence, quantity, and stock were consistently applied across all ordering paths, whether via direct orders or cart checkouts. Systematic testing using curl demonstrated that the endpoints produced correct JSON outputs for both normal and erroneous cases, and that stock levels and cart contents stayed synchronized with user actions. Overall, the exercise strengthened understanding of how to design and implement backend services that not only expose endpoints but also rigorously enforce domain-specific rules, providing a solid foundation for more complex transactional systems.

References

- Node.js Documentation – https://nodejs.org/
- Express.js Guide – https://expressjs.com/
- MDN Web Docs: HTTP Methods and Status Codes – https://developer.mozilla.org/
- Course-provided lab instructions and sample curl commands for the Product Ordering API